

## Welcome to IVISS.

IVISS is a multi-tenant platform for law-enforcement and regulatory teams to run roadside controls, verify vehicle compliance, and manage enforcement workflows.

This README is here to make your work safer, faster, and more effective. It is the entry point for developers, operators, and reviewers. It gives a concise map of the system, local setup, architecture boundaries, security model, API/testing conventions, and where to find the full detailed documentation.

## Table of Contents

1. [Overview](#)
2. [Architecture & Design Decisions](#)
3. [Tech Stack](#)
4. [Multi-tenancy & Security Model](#)
5. [Local Development](#)
  - 5.1 [Prerequisites](#)
  - 5.2 [Configuration \(`.env`\)](#)
  - 5.3 [Startup](#)
  - 5.4 [Useful Commands](#)
6. [Frontend Architecture](#)
7. [Backend Architecture \(Rust\)](#)
8. [Database & Data Model](#)
9. [API Reference & Conventions](#)
10. [Testing Strategy](#)
11. [Quality & Coding Standards](#)
12. [Debugging & Troubleshooting](#)
13. [Deployment & CI/CD](#)
14. [Monitoring & Logging](#)
15. [Contribution Guidelines](#)
16. [Appendices](#)

# 1. Overview

IVISS helps agencies:

- Identify vehicles (manual entry, photo/OCR workflows).
- Perform field control operations and keep traceable control history.
- Trigger and track enforcement actions.
- Operate with role-based access control in a multi-organization environment.
- Use a mobile-first web experience with PWA capabilities.

For system overview and functional context, see [docs/overview.md](#).

# 2. Architecture & Design Decisions

High-level architecture:

- **Frontend:** React + TypeScript + Vite (mobile-first SPA, PWA-enabled)
- **Backend:** Rust + Axum + SQLx (REST API + OpenAPI)
- **Database:** PostgreSQL 15
- **Infrastructure:** Docker Compose (local), Terraform + Ansible + GitHub Actions (deployment)

Main design choices:

- Multi-tenant data isolation by organization.
- Backend-generated OpenAPI contract consumed by frontend codegen.
- Role-based access control enforced in backend middleware.
- Infrastructure-as-Code and automated CI/CD pipelines.

For complete architecture diagrams and rationale, see: [docs/architecture\\_spec.md](#) and [docs/sequence\\_diagrams\\_&\\_flows.md](#)

# 3. Tech Stack

## Frontend

- React, TypeScript, Vite
- Tailwind CSS + shadcn/ui (Radix primitives)

- TanStack Query, React Router
- Vitest + Testing Library

## Backend

- Rust, Cargo, Axum, Tokio
- SQLx (PostgreSQL)
- Utoipa + Swagger UI (OpenAPI generation and docs)

## Platform

- Docker / Docker Compose
- GitHub Actions CI/CD
- AWS Lightsail (deployment target)

## 4. Multi-tenancy & Security Model

Core model:

- Organization-scoped data and workflows.
- RBAC across `admin`, `manager` (business label: supervisor), `org_admin`, and `agent` roles.
- JWT-based authentication with role-aware access middleware.
- Audit-oriented flows for sensitive operations.

Security notes:

- External providers (SMS/Email) are configured via environment variables.
- Secrets must never be committed.
- Authentication/session controls are handled server-side.

For full details, see: [docs/auth\\_tokens.md](#), [docs/developer/api.md](#) and [docs/developer/coding-standards.md](#)

## 5. Local Development

### 5.1 Prerequisites

- Docker Engine 20.10+
- Docker Compose v2+
- Node.js 20+ (for frontend local dev and tooling)

- Rust toolchain + Cargo (for backend local build/test)

## 5.2 Configuration ( `.env` )

```
cp .env.example .env
```

Minimum values for local backend startup:

- `POSTGRES_PASSWORD`
- `EXTERNAL_POSTGRES_PASSWORD`
- `JWT_PRIVATE_KEY_PEM`
- `JWT_PUBLIC_KEY_PEM`
- `ACTIVATION_CODE_PEPPER`
- `SMS_PROVIDER`

Recommended for end-to-end back-office flows:

- `EMAIL_PROVIDER` (mock, `resend`, `lettre/smtp`)
- Provider credentials for the selected email mode

Notes:

- The meaning of all those env variables are correctly documented in the `.env.example` file
- Real org-admin email delivery (temporary password) requires a real provider (`resend` or `lettre/smtp`) and valid credentials.

For the full setup matrix, see [docs/developer/getting-started.md](https://docs.developer/getting-started.md).

## 5.3 Startup

```
docker compose --profile dev up -d --build
```

Useful local URLs:

- Frontend: <http://localhost:8080>
- Backend health: <http://localhost:3000/api/v1/health>
- Swagger UI: <http://localhost:3000/docs>
- OpenAPI JSON: <http://localhost:3000/api-doc/openapi.json>
- Adminer: <http://localhost:8081>
- Metrics health: <http://localhost:9091/health>

## 5.4 Useful Commands

```
# Service status
docker compose ps
# Logs
docker compose logs -f backend
docker compose logs -f frontend
# Stop stack
docker compose down
# Stop + remove local volumes (destructive)
docker compose down -v
```

Frontend outside Docker (optional):

```
cd frontend
npm install
npm run dev
```

## 6. Frontend Architecture

Frontend structure is organized around routes, pages, reusable components, hooks, and generated OpenAPI client layers.

For the complete frontend architecture and developer breakdown, see: [docs/FE\\_ARCHITECTURE.md](#) and [docs/developer/project-structure.md](#)

## 7. Backend Architecture (Rust)

Backend is an Axum service with modular boundaries for handlers, services, queries, middleware, DTOs, and tests.

Key entry points:

- Runtime: `iviss-backend/src/main.rs`
- Routes: `iviss-backend/src/routes.rs`
- OpenAPI: `iviss-backend/src/api_doc.rs`

For complete backend structure and conventions, see: [docs/developer/project-structure.md](#) and [docs/developer/api.md](#)

## 8. Database & Data Model

IVISS uses PostgreSQL with SQLx migrations and seed mechanisms.

Main domains include:

- Organizations and users/roles
- Vehicle registry and statuses
- Control records and actions

- Pending submission workflow
- Audit logs

For full schema and DB workflow documentation, see: [docs/schema.md](#), [docs/data.md](#) and [docs/developer/database.md](#)

## 9. API Reference & Conventions

API contract is generated from backend code and exposed through OpenAPI.

References:

- Swagger UI (local): <http://localhost:3000/docs>
- OpenAPI JSON (local): <http://localhost:3000/api-doc/openapi.json>
- Frontend snapshot: `frontend/openapi.json`

API implementation and conventions are documented in: [docs/developer/api.md](#)

## 10. Testing Strategy

Testing is split by subsystem:

- Backend: Rust tests, integration tests, DB tests (Testcontainers)
- Frontend: unit/component tests with Vitest + Testing Library
- CI: workflow-based validation for build, lint, type checks, tests, coverage, and security scans

For full commands and CI mapping, see: [docs/developer/testing.md](#)

## 11. Quality & Coding Standards

Project standards cover:

- Code style and module ownership
- API/DB change discipline
- Security defaults
- Conventional commits and PR hygiene

See: [docs/developer/coding-standards.md](#)

## 12. Debugging & Troubleshooting

Use a layered troubleshooting approach:

- Service health and logs
- Env/config validation
- Auth/RBAC checks
- OpenAPI/codegen sync issues

See: [docs/developer/debugging.md](#)

## 13. Deployment & CI/CD

## Production deployment model:

- Infrastructure provisioning: Terraform
- Server configuration and rollout: Ansible + Docker Compose
- Automation: GitHub Actions
- Target: AWS Lightsail

For full deployment procedures and operational details, see: [docs/deployment\\_guide.md](#) and [docs/release\\_guide.md](#)

## 14. Monitoring & Logging

Operational observability includes:

- Prometheus scraping
- Grafana dashboards
- Service-level health endpoints and container logs

See: [docs/monitoring.md](#) and [monitoring/README.md](#)

## 15. Contribution Guidelines

- Use dedicated feature branches.
- Keep PRs focused and reviewable.
- Update docs whenever behavior, interfaces, or workflows change.
- Run relevant local checks before opening a PR.

Developer contribution standards: [docs/developer/coding-standards.md](#)

## 16. Appendices

### A) Project Structure Snapshot

```
iviss/
├── .github/
│   └── workflows/
├── docs/ # All documentation
│   ├── architecture_spec.md
│   ├── developer/
│   │   ├── README.md
│   │   └── ...
├── frontend/ # React Frontend
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── pages/
│   │   ├── router/
│   │   ├── services/
│   │   └── openapi-rq/
│   ├── public/
│   ├── package.json
│   ├── vite.config.ts
│   └── openapi.json
├── iviss-backend/ # Rust Backend
│   ├── src/
│   │   ├── handlers/
│   │   ├── services/
│   │   ├── queries/
│   │   ├── middleware/
│   │   ├── dto/
│   │   ├── tests/
│   │   ├── main.rs
│   │   ├── routes.rs
│   │   └── api_doc.rs
│   ├── migrations/ # SQL migrations
│   ├── seeds/
│   ├── scripts/ # Utility scripts
│   └── Cargo.toml
├── infra/ # Infrastructure as Code
│   ├── terraform/
│   ├── ansible/
│   └── scripts/
├── monitoring/ # Observability
│   ├── prometheus/
│   └── grafana/
├── scripts/ # Utility scripts
├── docker-compose.yml
├── .env.example
└── README.md
```

For detailed structure, see [docs/developer/project-structure.md](https://github.com/iviss/docs/developer/project-structure.md).

## B) Documentation Map

- [docs/overview.md](#)
- [docs/architecture\\_spec.md](#)
- [docs/deployment\\_guide.md](#)
- [docs/developer/README.md](#)
- [docs/developer/getting-started.md](#)
- [docs/developer/api.md](#)
- [docs/developer/database.md](#)
- [docs/developer/testing.md](#)
- [docs/developer/coding-standards.md](#)
- [docs/developer/debugging.md](#)

## C) License

This project is proprietary. All rights reserved.

## Document Version

**Version:** 1.0

**Last Updated:** May 04, 2026

**Author:** IVISS Development Team

For the latest version of this guide, check the Help section in the IVISS back-office or contact your system administrator.